ASSIGNMENT 3 – CO3

CSA16 – DATA WAREHOUSING AND DATA MINING

Student Name: M. Pranay

Register Number: 192525382

Department: Computer Science and Engineering

Q1. Decision Tree Induction: Choosing the Splitting Attribute (6 Marks)

Class and candidate attributes

Class attribute: Approved (Yes/No). Candidate splitting attributes: Income, Credit History and Employment. Stop a branch when it is pure, no useful attribute remains, or a minimum stopping condition is reached. Pruning removes branches that do not improve performance on unseen data.

Entropy and information gain

There are 12 records: 5 Yes and 7 No. Entropy(S) = $-(5/12 \log_2(5/12) + 7/12 \log_2(7/12)) = 0.9799$ bits.

Attribute	Weighted entropy	Information gain
Income	0.8113	0.1686
Credit History	0.7842	0.1957
Employment	0.9591	0.0207

Credit History has the highest information gain (0.1957), so it is selected as the root. Credit History = Poor contains 1 Yes and 5 No (entropy 0.6500). Credit History = Good contains 4 Yes and 2 No (entropy 0.9183).

Within the Good subset, Employment separates Salaried (3 Yes, 0 No) from Self-employed (1 Yes, 2 No), giving a larger gain than Income. For Good and Self-employed, Income separates High (Yes), Medium (No) and Low (No).

Gain ratio and Gini index

For Credit History, split information = 1.000, so Gain Ratio = 0.1957/1.000 = 0.1957. Weighted Gini after splitting on Credit History = (6/12 × 4/9) + (6/12 × 10/36) = 0.3889. Credit History remains the best root among the three attributes.

Decision tree and rules

- Credit History = Poor → Approved = No (majority class).

- Credit History = Good → check Employment.
- Good and Salaried → Approved = Yes.
- Good and Self-employed → check Income: High → Yes; Medium → No; Low → No.

Equivalent paths: (1) IF Credit History = Poor THEN Approved = No. (2) IF Credit History = Good AND Employment = Salaried THEN Approved = Yes. (3) IF Credit History = Good AND Employment = Self-employed AND Income = High THEN Approved = Yes. (4) IF Credit History = Good AND Employment = Self-employed AND Income is Medium or Low THEN Approved = No. The Poor branch is not pure because it contains one Yes record; the leaf predicts its majority class.

Weka validation and overfitting

In Weka, load the 12 records as ARFF/CSV, set Approved as the class, then select Classify → trees → J48. Record the tree, leaves, size, accuracy and parameter settings. Repeat using credit-g.arff. Change confidence factor (C) and minimum instances per leaf (M) one at a time.

Run	C	M	Tree size/leaves	Accuracy
12 tuples – default	0.25	2	Record from Weka	Record from Weka
12 tuples	0.10	2	Record from Weka	Record from Weka
12 tuples	0.40	2	Record from Weka	Record from Weka
credit-g	0.25	2	Record from Weka	Record from Weka

A smaller confidence factor generally causes stronger pruning; a larger minimum leaf size tends to simplify the tree. Overfitting occurs when a tree memorises training noise and performs worse on unseen data. A unique Customer_ID can create many branches and memorise rows, so it should normally be removed.

Q2. Bayesian Classification and the Cost of Assumptions (6 Marks)

Applicant X = (Income = Low, Credit History = Good, Employment = Self-employed). Class counts: Yes = 5 and No = 7.

Probability	Approved = Yes	Approved = No
Prior	5/12	7/12

P(Income=Low | class)	1/5	2/7
P(Credit=Good | class)	4/5	2/7
P(Self-employed | class)	2/5	4/7

Yes proportional posterior = (5/12)(1/5)(4/5)(2/5) = 0.02667. No proportional posterior = (7/12)(2/7)(2/7)(4/7) = 0.05442. Therefore, predict Approved = No.

Laplace correction

A zero conditional probability makes the class product zero. Laplace correction uses P(value | class) = (count + 1)/(class count + number of possible values).

Conditional probability	Yes corrected	No corrected
Income = Low (3 values)	2/8	3/10
Credit = Good (2 values)	5/7	3/9
Self-employed (2 values)	3/7	5/9

Corrected Yes score = (5/12)(2/8)(5/7)(3/7) = 0.03189. Corrected No score = (7/12)(3/10)(3/9)(5/9) = 0.03241. The prediction remains No, although the scores are close.

Weka and independence investigation

In Weka, load the loan data, set Approved as class and choose bayes → NaiveBayes. Use 10-fold cross-validation and display class probabilities if available. Compare Weka output with the manual probabilities.

Create a synthetic dataset with strongly correlated attributes, such as Monthly_Income and Annual_Income_Band, and a class label. Train Naïve Bayes and J48 on identical splits. Record accuracy with both correlated attributes and after removing one. Do not report a measured result until the experiment is run. Correlation may cause Naïve Bayes to count similar evidence more than once, but its class ranking can still be useful.

Naïve Bayes is useful for fast training and probabilistic predictions. A decision tree is preferable when attribute interactions and clear decision paths are important. Naïve Bayes may be less reliable when strong dependence distorts its probability estimates.

Q3. Rule-Based and Associative Classification (7 Marks)

IF–THEN rules and quality

- R1: IF Credit History = Poor THEN Approved = No.
- R2: IF Credit History = Good AND Employment = Salaried THEN Approved = Yes.
- R3: IF Credit History = Good AND Employment = Self-employed AND Income = High THEN Approved = Yes.
- R4: IF Credit History = Good AND Employment = Self-employed AND Income = Medium THEN Approved = No.
- R5: IF Credit History = Good AND Employment = Self-employed AND Income = Low THEN Approved = No.

Coverage is the number of records matched. Accuracy = correctly classified matched records / all matched records.

Rule	Matched records	Correct	Coverage of 12	Accuracy
R1	6	5	50.0%	83.3%
R2	3	3	25.0%	100%
R3	1	1	8.3%	100%
R4	1	1	8.3%	100%
R5	1	1	8.3%	100%

A tree represents decisions in a hierarchy; rules represent conditions and conclusions separately. If rules conflict, choose the higher-confidence, more specific or earlier rule. For example, a rule checking Credit History, Employment and Income may be prioritised over a general Credit History rule.

Sequential covering learns one rule at a time. It adds a rule that covers positive examples and few negative examples, removes the positive examples already covered, and repeats until a stopping condition is met.

JRip/PART and associative rules

In Weka, select rules → JRip or PART, load credit-g.arff and run 10-fold cross-validation. Save the rules, accuracy, confusion matrix and settings, then compare with J48. Exact output depends on Weka version and settings.

A class-association rule has form X → class. Support is the fraction of all records containing X and the class. Confidence is the fraction of records containing X that belong to the class. Minimum support 25% means at least 3 of 12 records; confidence must be at least 70%.

- Credit History = Good AND Employment = Salaried → Approved = Yes. Support = 3/12 = 25%; confidence = 3/3 = 100%. This rule qualifies.

Income = High AND Credit History = Good → Approved = Yes has support 2/12 = 16.7%, so it does not meet minimum support. For the final evidence, attach the actual output of the association-rule mining tool.

For explaining every rejection, a pruned decision tree or ordered rule list is suitable because the decision can be traced through explicit conditions. Associative rules discover patterns but may overlap and may not explain every applicant unambiguously.

Q4. Backpropagation: What Does the Network Learn? (7 Marks)

Sigmoid(z) = $1/(1 + e^{-z})$. Inputs x1 = 1, x2 = 0; target = 1.

Forward pass: Net(h1) = 1(0.2) + 0(0.4) − 0.4 = −0.2; h1 = sigmoid(−0.2) = 0.45017. Net(h2) = 1(−0.3) + 0(0.1) + 0.2 = −0.1; h2 = 0.47502. Net(o) = 0.45017(−0.3) + 0.47502(−0.2) + 0.1 = −0.13054; output = 0.46753.

Output delta = (1 − 0.46753)(0.46753)(1 − 0.46753) = 0.13256. Hidden delta h1 = 0.45017(1 − 0.45017)(−0.3)(0.13256) = −0.00984. Hidden delta h2 = 0.47502(1 − 0.47502)(−0.2)(0.13256) = −0.00661.

Update rule: new weight = old weight + learning rate × destination delta × input. Bias is updated using the destination delta.

Parameter	Old value	Updated value (approx.)
x1→h1	0.20000	0.19114
x2→h1	0.40000	0.40000
x1→h2	−0.30000	−0.30595
x2→h2	0.10000	0.10000
h1→o	−0.30000	−0.24628
h2→o	−0.20000	−0.14328

Bias h1	−0.40000	−0.40886
Bias h2	0.20000	0.19405
Bias output	0.10000	0.21930

After updating, output is approximately 0.496. Squared error before = ½(1 − 0.46753)² = 0.14176; after = approximately ½(1 − 0.496)² = 0.1270. Error decreases. Small random weights break symmetry so hidden units can learn different features; identical initial weights can make units learn the same representation.

In Weka, use functions → MultilayerPerceptron on a real dataset. Change learning rate, hidden units and epochs one at a time, using the same evaluation method.

Run	Learning rate	Hidden units	Epochs	Training accuracy	Test accuracy	Time
1	0.1	a	500	Measure	Measure	Measure
2	0.3	a	500	Measure	Measure	Measure
3	0.1	a+1	500	Measure	Measure	Measure
4	0.1	a	1000	Measure	Measure	Measure

Overfitting occurs when training accuracy rises while test accuracy falls or stops improving. Neural networks can model complex patterns but usually require more training and are less interpretable than decision trees.

Q5. Support Vector Machines: Margin, Kernels and Scaling (7 Marks)

Positive points: (3,3), (4,3), (4,4). Negative points: (1,1), (1,2), (2,1). Support vectors are (3,3), (2,1) and (1,2). A maximum-margin separator is x1 + x2 − 4.5 = 0, with w = (1,1), b = −4.5. The positive support vector has x1+x2 = 6; negative support vectors have x1+x2 = 3.

Distance from each support-vector boundary to the separator = 1.5/√2. Total margin width = 3/√2 = 2.121 units approximately. Removing a non-support vector normally does not change the separator; removing a support vector can change it.

Soft-margin C controls the penalty for errors. Large C penalises errors strongly and may overfit; small C permits violations and provides stronger regularisation. Kernels such as linear, polynomial and RBF allow different decision boundaries.

Use SMO in Weka or SVC in Python on a nonlinear dataset such as concentric circles. Compare linear, polynomial and RBF kernels. Vary C and kernel parameters such as RBF gamma. Record accuracy and support-vector count before and after standardising the attributes.

Kernel / setting	C	Parameter	Accuracy	Support vectors
Linear	Record	—	Measure	Measure
Polynomial	Record	Degree	Measure	Measure
RBF	Record	Gamma	Measure	Measure
RBF after scaling	Record	Gamma	Measure	Measure

Scaling prevents large-range attributes from dominating distance and dot-product calculations. SVMs are useful for high-dimensional data and complex boundaries, but require parameter tuning, scaling and may be expensive on large datasets. They are less interpretable than small trees.

Q6. Evaluating Classifiers and Predictors (7 Marks)

Given TP = 40, FN = 20, FP = 30, TN = 910; total = 1,000.

Measure	Formula	Calculation / result
Accuracy	(TP+TN)/Total	950/1000 = 95%
Error rate	(FP+FN)/Total	50/1000 = 5%
Sensitivity / Recall	TP/(TP+FN)	40/60 = 66.67%
Specificity	TN/(TN+FP)	910/940 = 96.81%
Precision	TP/(TP+FP)	40/70 = 57.14%
F-measure	2PR/(P+R)	61.54%

A classifier predicting Not Fraud for every transaction correctly classifies 940 non-fraud cases and misses all 60 fraud cases. Accuracy = 940/1000 = 94%, but fraud recall = 0%. This is why accuracy alone is misleading for imbalanced data.

Actual values = {12, 15, 20, 25}; predicted = {10, 18, 19, 30}. Absolute errors = {2, 3, 1, 5}; squared errors = {4, 9, 1, 25}. MAE = 11/4 = 2.75. MSE = 39/4 = 9.75. RMSE = $\sqrt{9.75}$ = 3.12 approximately. RMSE penalises large errors more strongly than MAE.

A ROC curve plots True Positive Rate (sensitivity) against False Positive Rate (1 − specificity) as the threshold changes. A curve nearer the top-left is better. AUC = 1 indicates perfect ranking; 0.5 indicates random ranking.

Holdout uses one train/test split and is simple but depends on the split. 10-fold cross-validation trains on nine folds and tests on one, repeating ten times. Bootstrap samples with replacement and evaluates on out-of-bag records; the .632 estimate combines training and out-of-bag error using 0.632 weighting. Use stratified splits for imbalanced data.

Method / split	Accuracy	Recall	Precision	ROC/AUC
Holdout seed 1	Measure	Measure	Measure	Measure
Holdout seed 2	Measure	Measure	Measure	Measure
Holdout seed 3	Measure	Measure	Measure	Measure
Holdout seed 4	Measure	Measure	Measure	Measure
Holdout seed 5	Measure	Measure	Measure	Measure
10-fold CV	Measure	Measure	Measure	Measure
Bootstrap .632	Measure	Measure	Measure	Measure

Compare J48, Naïve Bayes and SMO using identical stratified folds in Weka Experimenter. Use paired fold results and an appropriate statistical test. A small difference in average accuracy alone does not prove a meaningful difference. Plot a ROC curve for one classifier and report AUC.

Q7. Design a Classification Solution from First Principles (10 Marks)

The target is readmission within 30 days. Only 6% of patients are readmitted. A false negative costs ten times as much as a false positive. Doctors need understandable reasons for alerts. Attributes include age, diagnosis group, length of stay, prior admissions, lab results and discharge medications; some values are missing.

Stage 1 – Independent proposals

Student A – initial proposal

I would use a pruned decision tree because it produces understandable IF–THEN rules. I would impute numeric missing values using the training-set median and categorical values using the mode or a missing category. I would use stratified cross-validation, recall, precision, ROC-AUC and total cost. I would tune the threshold to reduce false negatives.

Student B – initial proposal

I would compare a cost-sensitive decision tree with a logistic regression baseline. I would clean inconsistent records, impute using training data only, encode categorical attributes and use stratified

resampling or class weights. I would evaluate recall, precision, ROC/PR curves and total cost with stratified 10-fold cross-validation. Tree paths would explain alerts.

Stage 2 – Compare the designs

Area	Student A	Student B	Joint decision
Model	Pruned tree	Cost-sensitive tree + logistic baseline	Interpretable cost-sensitive tree; compare baseline
Missing data	Median/mode	Training-only imputation	Impute inside each training fold
Imbalance	Threshold adjustment	Resampling/class weights	Compare weights/resampling and threshold
Evaluation	Recall, precision, AUC, cost	Recall, precision, ROC/PR, cost	Use recall, precision, AUC and cost
Explanation	IF–THEN path	Tree path and factors	Show path and key factors

Stage 3 – Joint final design

Choose a pruned, cost-sensitive decision tree and compare it with logistic regression as a baseline. Impute numeric values using the training-fold median and categorical values using mode or a missing category. Encode categories consistently and remove duplicate or invalid records. Compare class weights/cost-sensitive learning with resampling. Resampling must occur only inside training folds to avoid data leakage.

Tune the decision threshold using validation data and select the threshold that minimises 10FN + FP while retaining useful recall and precision. For every alert, show the tree path, patient values and risk category. The model supports clinical review and does not replace the doctor.

Stage 4A – Analytical validation

Expected cost = 10 × FN + FP. The following are illustrative examples, not measured hospital results:

Scenario	TP	FN	FP	TN	Cost
A – more alerts	50	10	100	840	10(10)+100 = 200

B – fewer alerts	40	20	40	900	10(20)+40 = 240

Scenario A has lower cost (200) than B (240), so A is preferred under the stated cost ratio. The actual choice must be based on measured validation confusion matrices. Accuracy is inappropriate as the main measure: predicting every patient as not readmitted gives about 94% accuracy while detecting no readmissions. Recall and total cost are more meaningful; precision and AUC/PR-AUC help assess false alerts and ranking.

Decision-tree strengths include interpretable paths and modelling nonlinear interactions. Limitations include instability with small data changes, overfitting and possibly lower predictive performance than complex models.

Stage 4B – Experimental validation

Use a public analogue dataset for hospital readmission or an imbalanced medical classification task. Implement a majority-class or unweighted-tree baseline and the final cost-sensitive tree. Use stratified cross-validation. Report recall, precision, ROC-AUC/PR-AUC, confusion matrix and total cost, with and without imbalance handling.

Model	Recall	Precision	ROC/PR-AUC	FP	FN	Total cost
Majority baseline	Measure	Measure	Measure	Measure	Measure	Calculate
Tree – no balancing	Measure	Measure	Measure	Measure	Measure	Calculate
Cost-sensitive / resampled tree	Measure	Measure	Measure	Measure	Measure	Calculate

Calculate cost as 10 × FN + FP for each result. Report mean values across folds and explain whether balancing improves recall and cost and whether it creates too many false positives.

Stage 5 – Cross-defence

Student A should explain which cross-validation results, recall, precision and cost measurements support the design. Student B should explain that the 10:1 ratio comes from the question: each false negative has ten cost units and each false positive one. Choose the threshold by calculating 10FN + FP for each candidate threshold and selecting the lowest validation cost.

Stage 6 – Joint evaluation

Accuracy-driven classifiers struggle because 94% of patients are not readmitted; a majority-class prediction can look accurate while missing nearly all high-risk patients. This design prioritises sensitivity and lower false-negative cost, accepting some additional false positives. It may fail if training data are unrepresentative, important variables are missing, labels are inaccurate or readmission patterns change. Monitor the model and involve clinicians.

Before choosing a classifier, examine dataset size, class balance, missing values, feature types, noise, attribute relationships, data quality, interpretability requirements, computational limits and the relative costs of errors.

Contribution Record

Complete this table according to the actual work performed. Both students remain responsible for understanding every question.

Question	Student A – primary contribution	Student B – primary contribution	Joint work
Q1	Manual entropy, gain and tree	Weka J48 validation	Compare and interpret
Q2	Naïve Bayes calculations	Weka verification and correlation study	Discuss assumptions
Q3	Tree rules and rule quality	JRip/PART and association rules	Recommend representation
Q4	Backpropagation calculation	Network experiments	Interpret learning
Q5	Hyperplane and margin	Kernel and scaling experiments	Select suitable conditions
Q6	Metrics and prediction	Estimation methods	Interpret evaluation

	errors	and model comparison	
Q7	Analytical cost validation	Experimental validation	Compare designs and final solution